



Announcements

- Lab 00 + Welcome Form: Released! Due Tue 1/27
 - On the fence about **Regular** vs. **Bridge** vs. **Video** discussion?
 - Choose **Regular/Bridge to be assigned a time** that works for you. Video students (the default) cannot attend in-person.
 - Switch section times/formats until Add/Drop deadline (**Week 4**)
 - **61C Scholars** Pilot Program (self-identify on welcome survey)
 - Scholars-specific regular discussion + other activities/socials
- **CONCURRENT ENROLLMENT**
 - Everyone can get in, but official roster enrollment will take longer
 - Do NOT email us at this time!
 - If by **Monday evening**, you cannot complete lab00 because you cannot sign up for an instructional account, email cs61c@berkeley.edu



CS61C

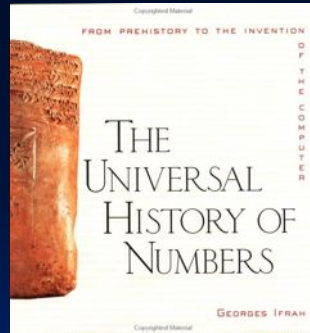
Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant Teaching
Professor
Lisa Yan

Number Representation

Great book $\Rightarrow$
The Universal History of Numbers
by Georges Ifrah



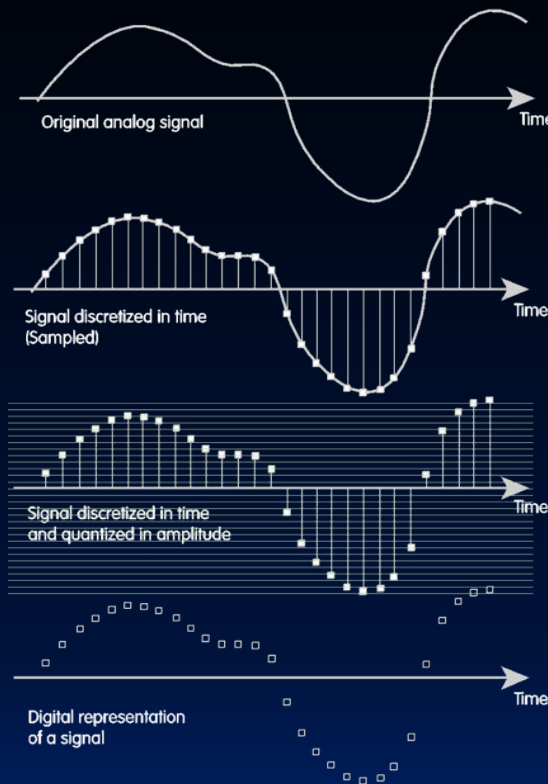
Yan, SP26

Data input: Analog $\rightarrow$ Digital

Real world is analog!

To import analog information, we must do two things:

- **Sample**
 - E.g., for a CD, every 44,100ths of a second, we ask a music signal how loud it is.
- **Quantize**
 - For every one of these samples, we figure out where, on a 16-bit (65,536 tick-mark) "yardstick", it lies.





Digital data not necessarily born Analog...





Discuss

1. How many “things” can be represented by 4 bits?

A. 4

B. 8

C. 16

D. 64

E. Something else

2. What does this particular 4-bit pattern represent?

1011

3. How many bits do you need to represent π (pi)?

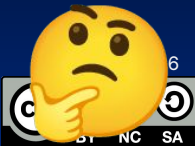
A. 1

B. 9 ($\pi=3.14$, so 0.011 “.” 001100)

C. 64 (Macs are 64-bit machines)

D. Every bit the machine has

E. ∞





BIG IDEA: Bits can represent anything!!

With N bits, you can represent at most 2^N things.

4 bits,
each 0 or 1

— — — —

1011

The meaning of a bitstring depends on the program context!

Today: Many ways to use N bits to represent numbers



Pop quiz??

1. [no pollEV, just discuss]
How many “things” can be represented by 4 bits?

A. 4

B. 8

C. 16

D. 64

E. Something else

2. [no pollEV, just discuss]
What does this particular 4-bit pattern represent?

1011

3. How many bits do you need to represent π (pi)?

A. 1

B. 9 ($\pi=3.14$, so 0.011”.” 001100)

C. 64 (Macs are 64-bit machines)

D. Every bit the machine has

E. ∞

BIG IDEA: Bits can represent anything!!

- Logical values? 1 bit
 - One possible convention: 0 → False, 1 → True
- Characters?
 - A, ..., Z: 26 letters → 5 bits ($26 \leq 32$)
 - ASCII: upper/lower case + punctuation → 7 bits → round to 1 byte
 - Unicode (www.unicode.com): standard code to cover all the world's languages ⇒ 8, 16, 32 bits
- Colors?
 - HTML color codes: 24 bits (3 bytes)
- Locations / addresses?
Commands?
 - IPv4 (32 bit), IPv6 (64 bit), etc.



California Gold

0xFDB515

Red (FD)

Green (B5)

Blue (15)



Agenda

Binary, Decimal, Hex

- Binary, Decimal, Hex
- Integer Representations
- Sign-Magnitude, Ones' Complement
- Two's Complement
- Bias Encoding



Number vs Numeral

Numeral

A symbol or name that stands for a number
e.g., 4 , *four* , *quattro* , IV , IIII , ...

...and **Digits** are symbols that make numerals

Above the abstraction line

Abstraction Line

Below the abstraction line

Number

The "idea" in our minds...there is only ONE of these
e.g., *the concept of "4"*



Decimal: Base 10 (Ten) #s

Digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Example:

$$\mathbf{3271} = \mathbf{3271}_{10} = (\mathbf{3} \times 10^3) + (\mathbf{2} \times 10^2) + (\mathbf{7} \times 10^1) + (\mathbf{1} \times 10^0)$$



Base 2 (Two) #s, Binary

Digits: 0, 1 (binary digits → bits)

Example: Binary number "1101"

Convert to decimal:

$$\begin{aligned} \text{0b1101} &= 1101_2 = (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ &= 8 + 4 + 0 + 1 \\ &= 13 \end{aligned}$$

Common binary shorthand:
0b1101



Discuss #2

1. How to represent 165 in binary?
2. Is this binary value human-readable?
What if it appeared multiple times in a program?

0b1110100101



Yan, SP26



Consider

- How to represent 165 in binary?

1 0 1 0 0 1 0 1

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

~~165~~ ~~37~~ 5 4 0



Base 16 (Sixteen) #s, Hexadecimal

Digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
10, 11, 12, 13, 14, 15

Example: Hexadecimal number "**A5**"

Convert to decimal:

$$\begin{aligned} \text{0xA5} &= \text{A5}_{16} = (10 \times 16^1) + (5 \times 16^0) \\ &= 160 + 5 \\ &= 165 \end{aligned}$$

"Hex" for short.
Common hex
shorthand: 0xA5



Hexadecimal is a Useful Shorthand for Binary (1/2)

Dec	Hex	Bin
00	0	0000
01	1	0001
02	2	0010
03	3	0011
04	4	0100
05	5	0101
06	6	0110
07	7	0111
08	8	1000
09	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Memorize this table.

Hexadecimal is a Useful Shorthand for Binary (2/2)

Dec	Hex	Bin
00	0	0000
01	1	0001
02	2	0010
03	3	0011
04	4	0100
05	5	0101
06	6	0110
07	7	0111
08	8	1000
09	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

The number 165

- In binary ("prefix 0b")

0b10100101



0xA5

- In hexadecimal (prefix "0x")



Back to Question 2 (1/2)

1. How to represent 165 in binary?

0b10100101

= 0xA5

2. Is this binary value human-readable?

What if it appeared multiple times in a program?

0b1110100101



Zero pad leftmost group of 4 bits as needed

1. How to represent 165 in binary?

0b10100101

= 0xA5

2. Is this binary value human-readable?

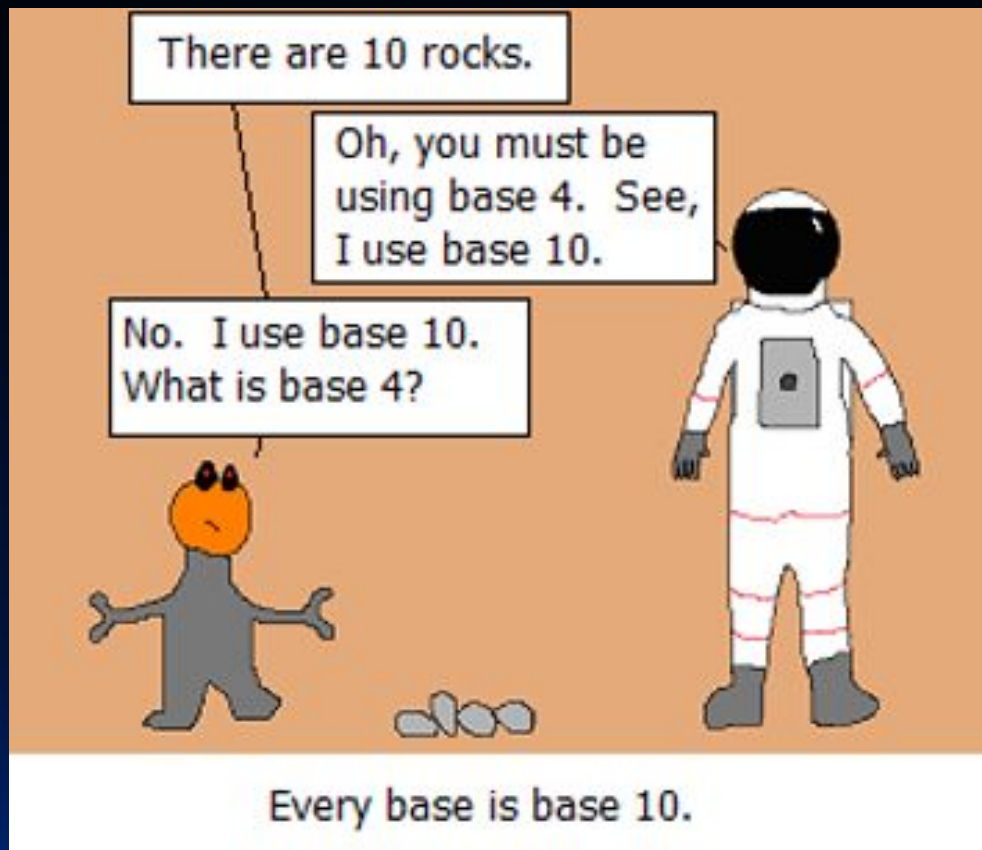
What if it appeared multiple times in a program?

0011
3

1010
A

0101
5

Which base do we use? (1/2)



How many
rocks?



Which base do we use? (2/2)

- **Decimal:** great for humans, especially when doing arithmetic
- **Hex:** if human looking at long strings of binary numbers, it's much easier to convert to hex and see 4 bits/symbol
 - Terrible for arithmetic on paper
- **Binary:** what computers use
 - **To a computer, numbers are always binary**
- Different reps of the same number:
 - $32_{\text{ten}} == 32_{10} == \text{0x20} == 100000_2 == \text{0b}100000$



Integer Representations

- Binary, Decimal, Hex
- Integer Representations
- Sign-Magnitude, Ones' Complement
- Two's Complement
- Bias Encoding



How do we pick a representation for integers? (1/2)

- Want a representation that supports common integer operations:
 - Add them
 - Subtract them
 - Multiply them
 - Divide them
 - Compare them ($<$, $=$, $\neq$, $\leq$, etc.)



N-Bit Unsigned Integer Representation

- Most computers use the **unsigned** integer representation from mathematics (that we just discussed.)
 - C's `uint8_t`, `uint16_t`, etc.: $[0, 2^N-1]$

- Example: $10 + 7 = 17$
 - 10, 7 can be represented with 4 bits:
 - Addition, subtraction just as you would in decimal!!
 - So simple to **add** in binary that we can build circuits to do it!
 - **This design decision would make hardware simple!**
 - ...wait...

$$\begin{array}{r} 11 \text{ carry bits} \\ 1010 \\ + 0111 \\ \hline 10001 \end{array}$$

How do we pick a representation for integers?(2/2)

- While strictly speaking, binary numerals have an infinite number of digits, **hardware has limits**.
 - Must also specify the number of bits for an integer representation.
- An N-bit representation can only represent 2^N things.
When doing arithmetic, numbers might "wrap around."



Car odometer

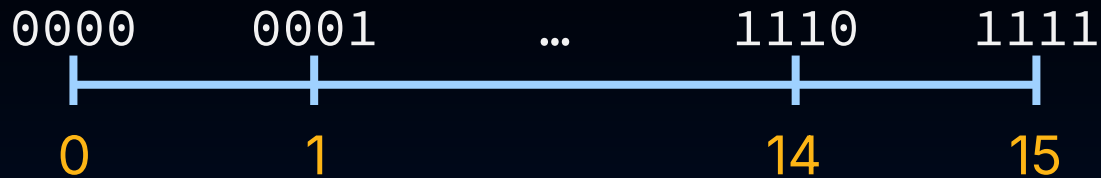
With a 4-bit
unsigned integer
representation,
 $10 + 7 = 1$.

$$\begin{array}{r}
 11 \text{ carry bits} \\
 1010 \\
 + 0111 \\
 \hline
 10001
 \end{array}$$

dropped



Number line vs. number "wheel"



Integer overflow: The arithmetic result is outside the representable range.

Number wheels visualize integer overflow.





Sign-Magnitude, Ones' Complement

- Binary, Decimal, Hex
- Integer Representations
- Sign-Magnitude, Ones' Complement
- Two's Complement
- Bias Encoding

Let's move to discussing **signed** integer representations, which can represent positive and negative #s.



N-Bit Signed Integer Representations

For the proposed N-bit signed integer representations, consider:

1. How to represent 5? Is it easy to convert to -5?
2. Is it easy to identify positive vs. negative from a bit pattern?
3. Is addition easy to implement? Example: $5 + -5 = 0$
4. How to represent zero?
5. [at home] What is the positive integer range? Negative integer range?

Ain't no free lunch: We still have N bits.
Goal: half positive, half negative, and zero



Two “strawman” designs for N-bit signed integer rep (1/2)

- Sign-Magnitude
 - Leftmost bit is **sign bit**
 - Remaining N-1 bits is magnitude, i.e., numerical value

$$+5_{\text{ten}} = 0b \ 0000 \ 0101$$

$$-5_{\text{ten}} = 0b \ 1000 \ 0101$$



Two “strawman” designs for N-bit signed integer rep. (2/2)

- One's Complement
 - (reserve leftmost bit for sign)
 - If positive, unsigned rep
 - If negative, **flip** bits of positive rep

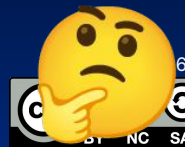
$$+5_{\text{ten}} = 0b \ 0000 \ 0101$$

$$-5_{\text{ten}} = 0b \ 1111 \ 1010$$

Discuss #3

- Sign-Magnitude
 - Leftmost bit is **sign bit**
 - Remaining N-1 bits is magnitude, i.e., numerical value
- One's Complement
 - (reserve leftmost bit for sign)
 - If positive, unsigned rep
 - If negative, **flip** bits of positive rep

1. How to represent 5? -5? [done]
2. How to represent zero?
3. Is it easy to identify positive vs. negative?
4. Is addition easy to implement? Example: $5 + -5 = 0$

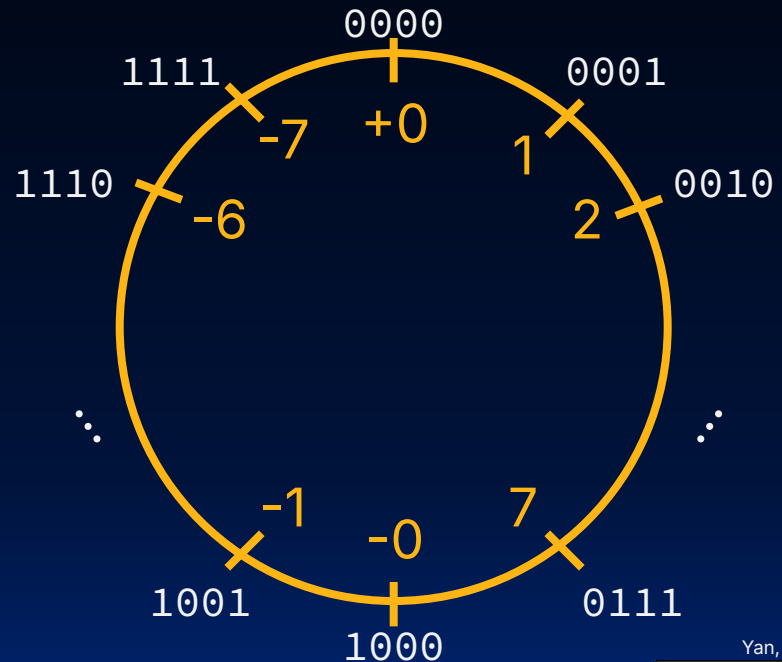


Sign-Magnitude: Rarely Used

- Sign-magnitude is **rarely used**, due to many shortcomings:
 - Incrementing "binary odometer" increases then decreases values
 - Arithmetic circuit complicated: depends on signs same/different
 - **Two zeros** (how to compare??)
- Reasonable for signal processing, not for general purpose computers

$$+5_{\text{ten}} = 0b \ 0000 \ 0101$$

$$-5_{\text{ten}} = 0b \ 1000 \ 0101$$

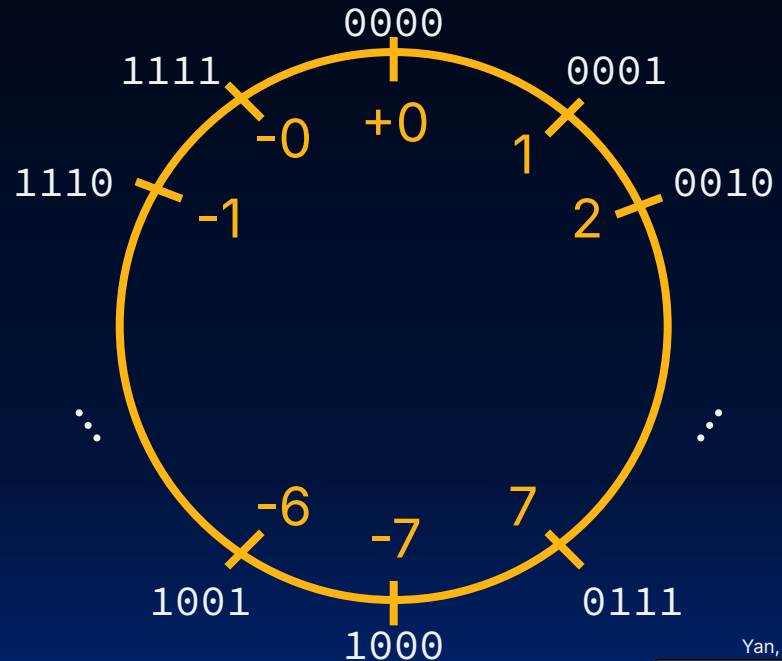


One's Complement: Old but OK

- Advantages:
 - Leftmost bit ("**most significant bit**") is still effectively sign bit
 - Incrementing binary odometer consistent on the # line
- Some disadvantages still persist:
 - Still two zeros
 - Arithmetic still somewhat complicated
- While used for a while on some computer products
 - It's not currently used in current hardware

$$+5_{\text{ten}} = 0b \ 0000 \ 0101$$

$$-5_{\text{ten}} = 0b \ 1111 \ 1010$$





Two's Complement

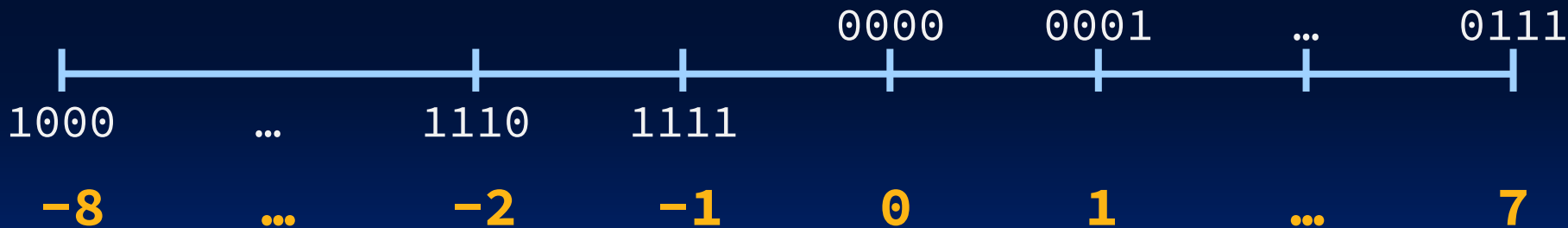
- Binary, Decimal, Hex
- Integer Representations
- Sign-Magnitude, Ones' Complement
- Two's Complement
- Bias Encoding

Two's Complement: The Motivation

- Ones' complement:

			0000	0001	...	0111
1000	...	1110	1111			
-7	...	-1	±0	1	...	7
- The problem: "Overlap" creates two 0s.

- The solution: **Shift the negative mappings left by one.**

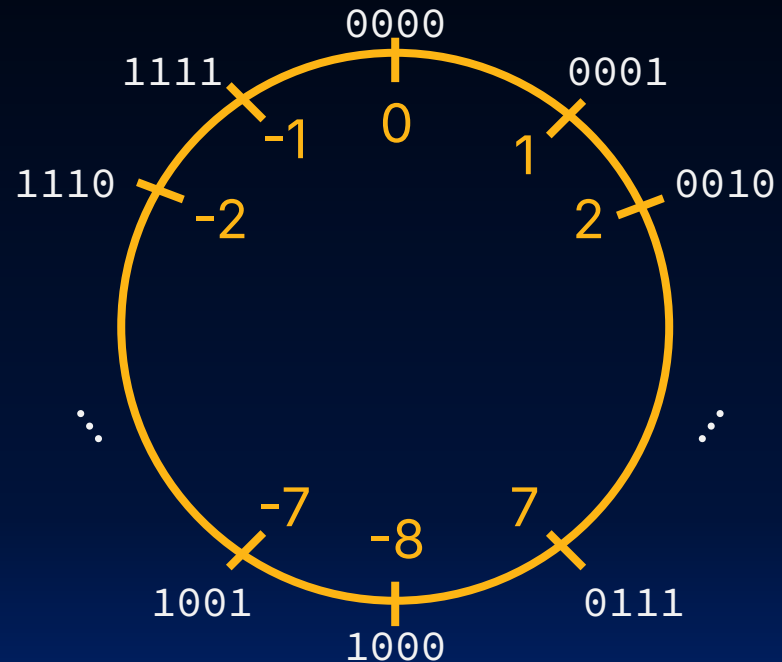


Two's Complement: Why?

- Advantages:
 - Leftmost bit ("**most significant bit**") is still effectively sign bit
 - One zero
 - **Simple hardware for addition**
 - Incrementing binary odometer consistent on the # line
- Two's complement is the representation used for all C23 signed integers.
 - `int8_t`, `int16_t`, etc.

$$+5_{\text{ten}} = 0b\ 0000\ 0101$$

$$-5_{\text{ten}} = 0b\ 1111\ 1011$$





Two's Complement: Procedural Algorithm

- Two's Complement
 - If positive, unsigned rep
 - If negative:
 - Start from positive rep
 - **flip** bits
 - **add one**

5_{ten} **0101** $\rightarrow$ 1010 $\rightarrow$ 1011 -5_{ten}

- Negative $\rightarrow$ positive is same procedure: flip and add one.
 - Makes hardware easy!

-5_{ten} **1011** $\rightarrow$ 0100 $\rightarrow$ 0101 5_{ten}

In notes: Read the singular mathematical formula for both positive and negative

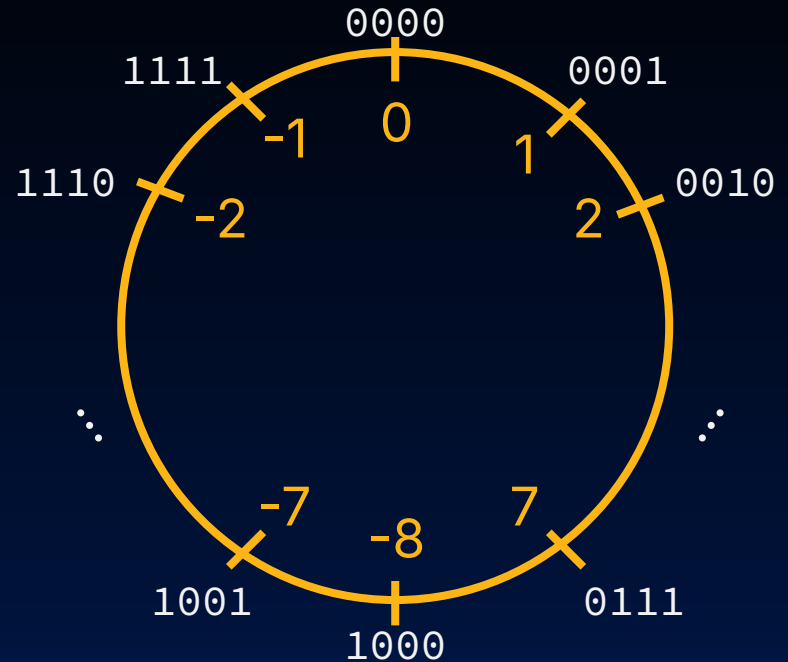
On your own: Prove math formula is equivalent to this slide's algorithm



Two's Complement: Addition

$$\begin{array}{r} \text{Decimal} \quad 5 \\ + \quad -5 \\ \hline 0 \end{array}$$

$$\begin{array}{r} \text{Two's complement} \quad 111 \text{ carry bits} \\ \quad 0101 \\ + \quad 1011 \\ \hline \text{dropped } 10000 \end{array}$$



Integer overflow in two's complement is a **feature!**
It means we can use the same HW to add numbers of different signs.

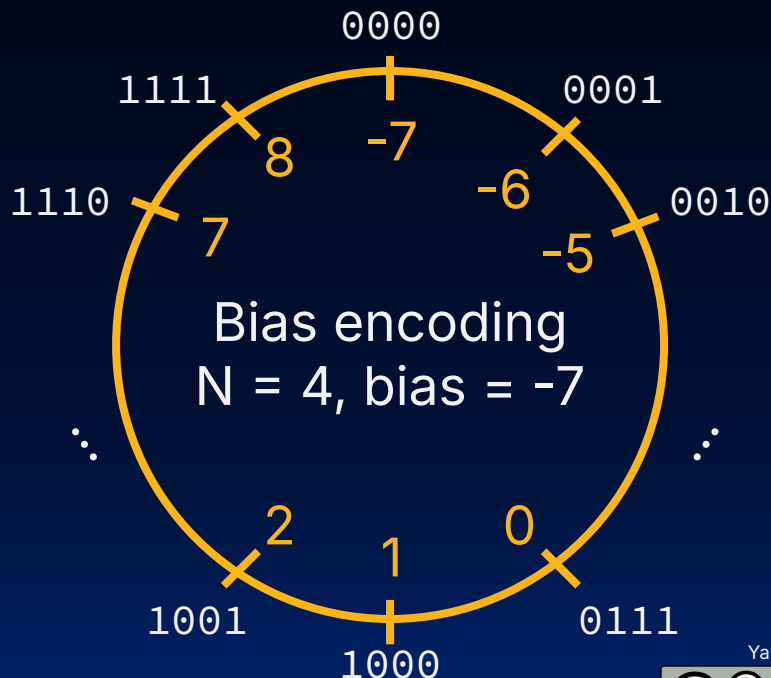
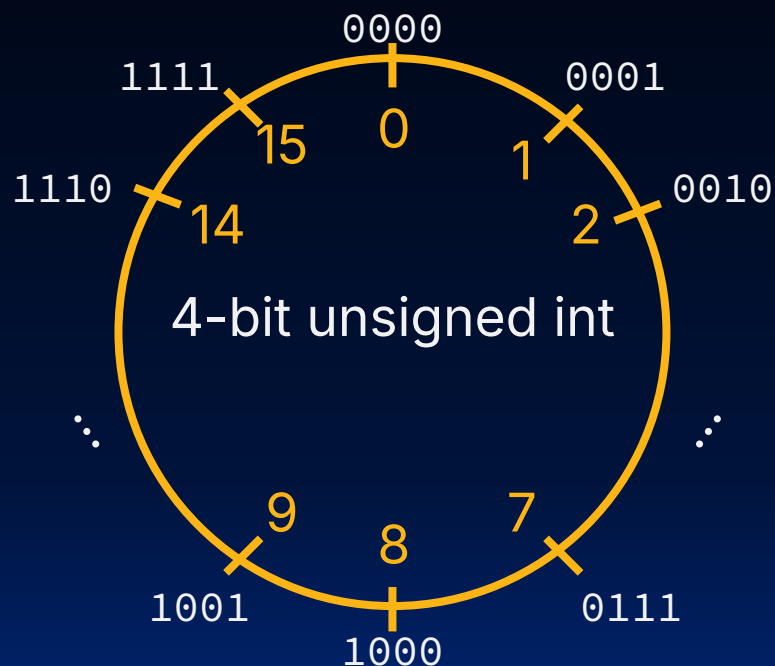
Bias Encoding

[at home] Used for
certain applications

- Binary, Decimal, Hex
- Integer Representations
- Sign-Magnitude, Ones' Complement
- Two's Complement
- Bias Encoding

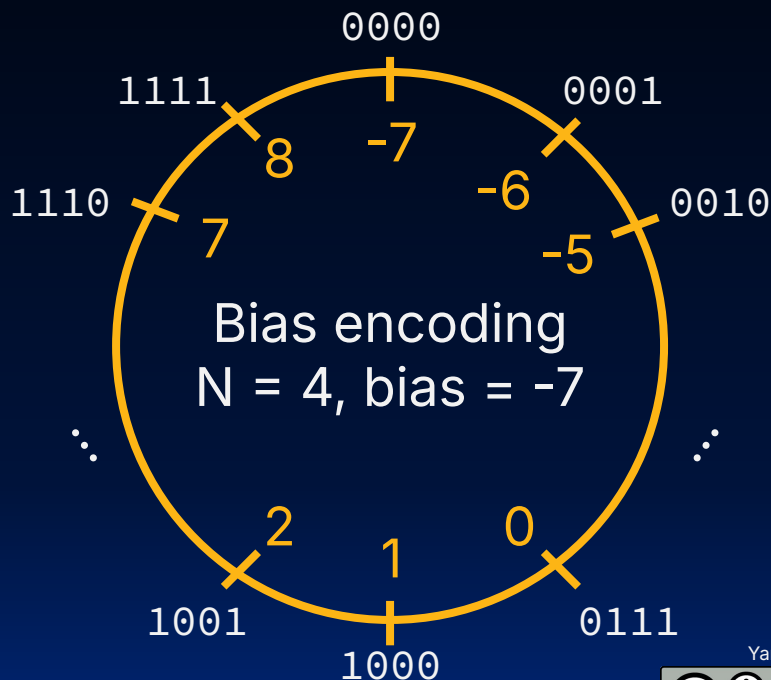
Bias Encoding

- **Bias encoding:** "Shift" the numbers to represent a target range of integers



Bias Encoding

- **Bias encoding:** "Shift" the numbers to represent a target range of integers
- $\text{Number} = (\text{unsigned rep}) + (\text{bias})$
 - Define a "bias"
 - To interpret stored binary: Read the data as an unsigned number, then add the bias
 - To store a data value: Subtract the bias, then store the resulting number as an unsigned number

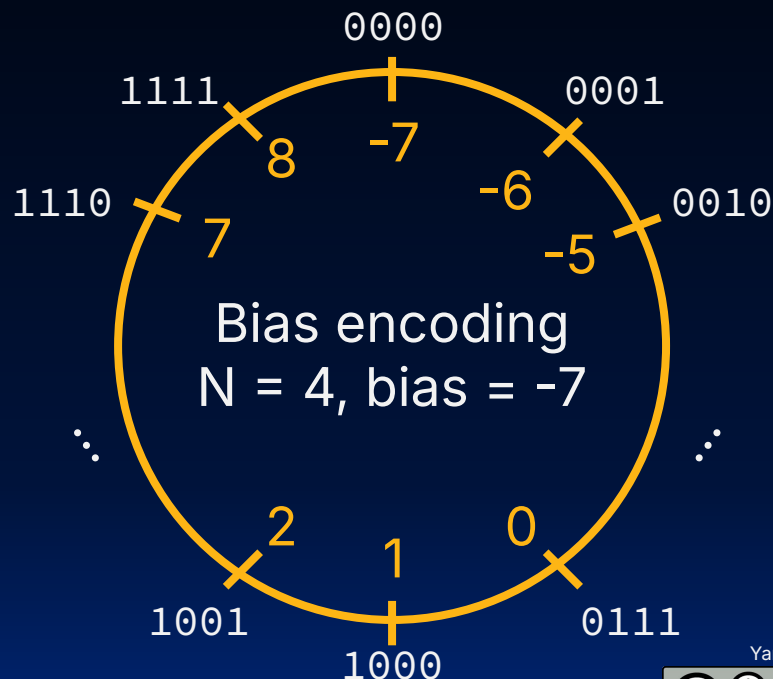


When is Bias Encoding Used?

- Number = (unsigned rep) + (bias)
- Useful for, say, AC signals with a DC offset
 - e.g., for 8V and 10V;
just store 2V range

Bias could be anything we want!

- In practice, with N bits, reasonable choice is $\text{bias} = -(2^{N-1} - 1)$
 - E.g., 4 bits
 $\text{bias} = -(2^3 - 1) = -(8 - 1) = -7$
- Used for floating point (more later)



Amazing Illustrations by Ketrina (Yim) Thompson

Binary Representations for Integers

In the early days of computing, designers made computers express numbers using **unsigned binary**. And they were content...

Until there were negative numbers.

To include negative numbers, designers came up with **sign magnitude**. That took care of the negative numbers...

But the computer had to count backwards for the negative numbers.

Plus, this introduced positive and negative zero.

Then designers created **one's complement**. Now computers only had to count in one direction.

But there were still two zeroes!

Finally, designers developed **two's complement**. Now, there was only one zero... And they were content.

Comparing Integer Representations

Negatives and Zeros

Here, we'll choose who gets to be the world's standard for computer integers. For now, let's introduce our contestants.

Round 1 - Negation

Round 2 - Zeros

Comparing Integer Representations

Increments and Monotonicity

Round 3 - Incrementing

On to Round 3! Using this board, graph how your value changes when you increment your bit pattern from 00000000 to 11111111. We'll give each graph a point for having a continuous graph and a point for a constant unit slope.

Very nice! Unsigned has a graph that is continuous and has a unit slope. This means we can use an unsigned comparator to compare integers! We'll give Unsigned two points for that.

What's that? Sign Magnitude has a very unusual increment indeed. It has a unit slope for positive integers, but the slope becomes -1 for negatives. Sorry, but no points for Sign Magnitude this round.

There's a discontinuity in the graph for One's Complement, but we do like how it has a constant unit slope. That's one point for One's Complement.

Just like One's Complement, Two's Complement has a discontinuous graph and unit slope. So we'll give Two's Complement a point.

Another monotonically increasing graph with unit slope! You can use an unsigned comparator to compare integers here, too. Bias gets two points!

Comparing Integer Representations

The Thrilling Conclusion

	Negation	One Zero?	Zero's 00000000	Continuous?	Monotonically Increasing?
Unsigned					
Sign Magnitude	✓		✓	✓	✓
One's Complement	✓		✓	✓	✓
Two's Complement	✓		✓	✓	✓
Bias	✓		✓	✓	✓

Well, well! It appears we have a three-way tie among Unsigned, Two's Complement, and Bias! We can certainly give each of our winners a prize, though.

Unsigned, you'll be the representation for data whenever users call upon the unsigned modifier in C! I've heard that other languages use it, too, so you'll work for them as well.

signed char foo = 242;

00011000

Bias, you'll represent the exponent in IEEE-754 Floating-point numbers! The fact that we can compare exponents with an unsigned comparator will come in handy!

And you, Two's Complement, because you can negate and have one zero that is expressed as all zero bits, you will be the representation of integers for binary computers all around the world!

See course notes for reference links!

And in summary...

- We represent “things” in computers as particular bit patterns:
 - With N bits, you can represent at most 2^N things.
 - Today, we discussed five different encodings for integers:
 - Unsigned integers
 - Signed integers:
 - Sign-Magnitude
 - Ones' Complement
 - Two's Complement
 - Bias Encoding
 - Computer architects make design decisions to make HW simple
 - Unsigned and Two's complement are C standard. Learn them!!
 - Integer overflow: The result of an arithmetic operation is outside the representable range of integers.
 - Numbers have infinite digits, but computers have finite precision.
- This can lead to arithmetic errors. More later!

For you to consider:
How could we represent -12.75?